

LESSON GUIDE

Technology Plus

Unit 5: Software

3.1.3 – Operating System Components

Lesson Overview:

In this lesson, students will explore the hidden parts of the operating system that keep computers running smoothly beyond files and folders.

Students will:

- Identify the role of OS components – including utilities, services, processes, and drivers – in keeping a system functional.
- Compare GUI and CLI interactions and explain how file attributes and properties affect file behavior.
- Investigate real system examples of processes, services, and drivers in Windows Server and Ubuntu, and explain how they connect to support usability and performance.

Guiding Question: How do an operating system's utilities, services, processes, drivers, and interfaces work together to keep the system functional, secure, and user-friendly?

Suggested Grade Levels: 8-12

Technology Needed: Yes

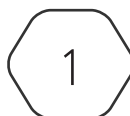
Approximate Time Required: 60-75 minutes

Standards:

CompTIA Tech+ FC0-U71 Objective

- 3.1 – Identify components of an OS.
 - System applications and utilities
 - Services
 - Processes
 - Drivers
 - Interfaces
 - Console/command line
 - Graphical user interface (GUI)
 - File attributes and properties

This content is based upon work supported by the US Department of Homeland Security's Cybersecurity & Infrastructure Security Agency under the Cybersecurity Education Training and Assistance Program (CETAP).



CYBER.ORG
THE ACADEMIC INITIATIVE OF THE CYBER INNOVATION CENTER

Copyright © 2025 Cyber Innovation Center
All Rights Reserved. Not for Distribution.

Lesson Background

Background Information:

Students usually think of an operating system as just “the screen you click on.” In reality, OS components extend far beyond the desktop. Utilities let users monitor and configure systems; services and processes quietly keep everything running; drivers make sure hardware works; and interfaces let us interact with the OS in both visual and command-line ways. Understanding these pieces gives students the context to troubleshoot issues, appreciate system security, and use the right tool for the right job. In this lesson, students will see these components in action on both Windows Server and Ubuntu to better understand how they support system performance.

Materials Needed:

Materials per Class	• Lesson Guide • Lesson Slides 3.1.3 – OS Components • Lab Slides 3.1.3 – OS Under the Hood
Materials per Student	• Student Notes 3.1.3 – OS Components • Student Handout 3.1.3 – OS Components • Assessment 3.1.3 – OS Components (optional)

Lesson Background

Cyber Terms:

Term	Definition
utility	a built-in OS tool for system monitoring or maintenance
service	a background program supporting the OS and apps
daemon	a service process that runs in the background (Linux/macOS)
process	a running instance of a program
context switching	the OS swapping between processes so multitasking is possible
device driver	software that lets the OS communicate with hardware
interface	how a user interacts with the OS (GUI or CLI)

3.1.3 – OS Components

Guiding Question: How do an operating system's utilities, services, processes, drivers, and interfaces work together to keep the system functional, secure, and user-friendly?

Lesson Launch (5-7 minutes)

- **Ask:** "What do you do when your computer runs really slowly?"
Student answers may include restarting, closing apps, or wanting to throw it out the window.
- Transition: These frustrations usually come back to processes, services, drivers, and utilities. The OS has built-in ways to monitor and fix them.

Putting the Components Together (15-20 minutes)

- Distribute the Student Handout 3.1.3 – OS Components and have students fill in the answers during the lesson.

System Applications & Utilities

- Explain: **Utilities** are **built-in tools** that help keep the system running – not apps you download.
- Examples:
 - **Task Manager / Activity Monitor / top** → show resource use, end frozen programs.
 - **File Explorer / Finder** → navigate files/folders.
 - **Settings / Control Panel** → adjust system options.
 - **Disk/Backup/Security tools** → check disk health, restore files, clean up junk, block malware.
- **Ask:** "Which of these do you think you've used before? What did you use it for?"

Services

- Explain: **Services** are **background programs** that start automatically and support the OS. Users don't launch them – but they're critical.
- Examples: Windows Update, Print Spooler, DNS Client, antivirus scanning, cron jobs.
- Note: In Linux/macOS, services are called **daemons**, a term borrowed from MIT in the 1960s. Inspired by Maxwell's demon, the idea is an invisible helper that tirelessly works behind the scenes.
- **Ask:** "What might happen if the Print Spooler service failed?"

Students may say, "The printer wouldn't print."

Processes

- Explain: A **process** = **a program in execution**. When you open an app, the OS makes one or more processes.
- Processes have **states**: New, Ready, Running, Waiting, Terminated.
- The OS uses **context switching** to share CPU time among processes, giving the appearance of

multitasking.

- Scheduling: Round Robin, First-Come-First-Served, Shortest Job Next, Priority.
- **Ask:** “Why do you think your Task Manager shows dozens of processes even when you only have one app open?”

Students may say, “Because the Task Manager reflects everything the CPU is actually juggling, not just the app window you see.”

Drivers

- Explain: **Drivers are translators** between the OS and hardware. Without them, the OS doesn’t know how to use your devices.
- Examples: printer drivers, graphics drivers, network drivers.
- Types:
 - **Kernel-mode** → fast, privileged, direct hardware access, but riskier.
 - **User-mode** → safer, indirect, less powerful.
- **Ask:** “Why might updating your graphics driver improve gaming performance?”

Students may say, “Updating the graphics driver can improve gaming performance by unlocking new features, improving efficiency, fixing bugs.”

Interfaces

- Explain: **Interfaces = how users interact with the OS.**
- **GUI** → visual, intuitive, icons/menus (Windows Explorer, Finder, GNOME).
- **CLI** → text commands, precise/powerful (Command Prompt, PowerShell, bash).
 - Examples: **dir/ls, cd, ipconfig/ifconfig.**
- File attributes (hidden, read-only, system) and file properties (size, type, timestamps) control how files are treated.
- **Ask:** “Have you ever tried to open a file and it wouldn’t let you? What attribute or property might cause that?”

Students may say, “The file’s permissions, attributes, or properties could prevent access. For example, if you don’t have the right permissions (like Read access), the OS won’t let you open it. Or, if the file is marked as hidden or system, the OS might block or limit access to protect it.”

Lab: OS Under the Hood (25-30 minutes)

- Display Lab Slides 3.1.3 – OS Under the Hood, have students record their answers in the handout.

Part A – Processes: Check Task Manager/top. Record running processes.

- In both machines process, CPU usage, and Memory will fluctuate. Instruct students to record whichever process they see in the moment.
- In Ubuntu:
 - Students will want to use Full Screen mode to better see the process information in the terminal

window.

- If students run **top** and press **q** to quit, the list stops fluctuating and is more of a snapshot view.
- If students run **htop**, they will see a more color presentation of the processes that are running, but once they press **q** or F10 to quit, the view goes back to the command line.
- Have students sort by %CPU or %MEM in **htop** by using arrow keys + F6 (sort menu). That way they can easily answer “which process is using the most CPU/Memory” without hunting.

Part B – Services: Identify one network service + one security/update service.

- **Keywords to Windows Service Startup Types**

- **Automatic** – Starts every time Windows starts. Always running.
- **Automatic (Trigger Start)** – Starts automatically **only when needed** (triggered by an event).
- **Manual** – Does **not** start on its own. Windows or an app can start it if needed.
- **Disabled** – Cannot start at all unless re-enabled by an administrator.
- Emphasize: **Status ≠ Startup** Type. A service may be set to *Manual* but currently *Running* because something triggered it.
- In Ubuntu, the SUB column is key:
 - running = continuous service.
 - exited = one-time service (it finished its job).

Part C – Drivers: Find network and display drivers in Device Manager.

- In **Windows Server 22 (VM)**:
 - The NIC driver will likely show as *Amazon Elastic Network Adapter*.
 - Display & mouse will show *Microsoft Basic* drivers (generic).
 - Point out: *This is because we’re on a virtual machine – only the NIC is AWS-specific hardware, while graphics/mouse don’t need specialized drivers in this environment.*
 - Students should check the **Driver tab** in Properties for: *Provider, Date, and Version.*
- In **Ubuntu**:
 - **lsmod** shows kernel modules (messy). Instead, use:
 - **sudo lshw -C network** → NIC driver info.
 - **sudo lshw -C display** → graphics driver info.
 - **sudo lshw -C input** → keyboard/mouse.
- Quick teacher fallback: If output scrolls too fast, tell students they can **pipe commands to less** (e.g., **sudo lshw -C network | less**) so they can scroll at their own pace.

Putting It All Together Why do we need all of these pieces working together?

- Why do you think your Task Manager (or top) showed so many processes, even when you only had one app open?

Because the operating system is running dozens of background processes and services in addition to the app window. Many of these support networking, graphics, and system stability.

- Why might some Windows services be set to *Manual* or *Disabled* instead of *Automatic*?

To save resources and improve security. Not every service is needed all the time, so Windows

only starts some when triggered. Others are disabled if they could pose risks or aren't required on most systems.

- Why is the network adapter driver in the Range provided by Amazon Web Services, while the display and mouse use Microsoft's basic drivers?

Because the lab machine is a virtual machine. The NIC is virtual hardware created by AWS, so it has its own AWS driver. The display and mouse aren't real hardware in a VM, so generic Microsoft drivers are enough to make them work.

Putting It All Together (5-10 minutes)

- Review what students learned today by discussing the questions below as a class.
- Why does the OS need both visible utilities and invisible services?

Utilities give users direct control (like Task Manager, File Explorer, or Disk Cleanup), while services handle essential background tasks automatically (like Windows Update or DNS Client). Together, they balance **user control** and **behind-the-scenes stability**.

- How do services and processes differ?

A **process** is any running program (visible or hidden). A **service** is a special kind of background process that usually starts at boot and runs without user interaction. So: all services are processes, but not all processes are services.

- Why is context switching important for multitasking?

Context switching allows the CPU to quickly switch between processes by saving one's state and loading another. This creates the illusion of smooth multitasking and prevents one app from hogging the CPU. Without it, a single process could freeze the whole system.

- What role do drivers play in making hardware usable?

Drivers are software translators between the OS and hardware. They let the OS "speak the language" of devices like printers, GPUs, and network cards. Without drivers, hardware is just an inert component – the OS wouldn't know how to use it.

- When would CLI be more effective than GUI?

CLI is more effective for **advanced, repetitive, or large-scale tasks** – like scripting updates on multiple servers, running diagnostics, or quickly copying lots of files. GUI is better for everyday, one-off tasks.

- How do all these pieces connect when running a simple app like a web browser?

The OS uses:

- **Drivers** → network card driver to connect online, graphics driver to display the browser.
- **Processes** → one for the browser, others for each tab/extension.
- **Services** → networking services like DNS to resolve websites.

- **Utilities** → Task Manager could show CPU/memory usage.
- **Interface** → GUI lets you point-and-click in the browser, CLI could show the same network traffic.

Together, they make a complex task (loading a webpage) feel simple to the user.